

DOSSIER TECHNIQUE FINAL

Plateforme de dématérialisation des appels à projets et gestion des subventions FDUIC

Auteur : Fatou Biaye

Profil : Développeur Web & Mobile

1. Contexte et Problématique

Le système actuel repose sur des méthodes manuelles. Cela entraîne des pertes de temps, des erreurs humaines, un manque de transparence et des difficultés de suivi. La transformation digitale devient donc nécessaire. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système actuel repose sur des méthodes manuelles. Cela entraîne des pertes de temps, des erreurs humaines, un manque de transparence et des difficultés de suivi. La transformation digitale devient donc nécessaire. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système actuel repose sur des méthodes manuelles. Cela entraîne des pertes de temps, des erreurs humaines, un manque de transparence et des difficultés de suivi. La transformation digitale devient donc nécessaire. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système actuel repose sur des méthodes manuelles. Cela entraîne des pertes de temps, des erreurs humaines, un manque de transparence et des difficultés de suivi. La transformation digitale devient donc nécessaire. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système actuel repose sur des méthodes manuelles. Cela entraîne des pertes de temps, des erreurs humaines, un manque de transparence et des difficultés de suivi. La transformation digitale devient donc nécessaire. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système actuel repose sur des méthodes manuelles. Cela entraîne des pertes de temps, des erreurs humaines, un manque de transparence et des difficultés de suivi. La transformation digitale devient donc nécessaire. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système actuel repose sur des méthodes manuelles. Cela entraîne des pertes de temps, des erreurs humaines, un manque de transparence et des difficultés de suivi. La transformation digitale devient donc nécessaire. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système actuel repose sur des méthodes manuelles. Cela entraîne des pertes de temps, des erreurs humaines, un manque de transparence et des difficultés de suivi. La transformation digitale devient donc nécessaire. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

2. Objectifs du projet

L'objectif principal est de digitaliser entièrement le processus. Les objectifs incluent la rapidité, la transparence, la sécurité et la centralisation des données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

L'objectif principal est de digitaliser entièrement le processus. Les objectifs incluent la rapidité, la transparence, la sécurité et la centralisation des données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

L'objectif principal est de digitaliser entièrement le processus. Les objectifs incluent la rapidité, la transparence, la sécurité et la centralisation des données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

L'objectif principal est de digitaliser entièrement le processus. Les objectifs incluent la rapidité, la transparence, la sécurité et la centralisation des données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

L'objectif principal est de digitaliser entièrement le processus. Les objectifs incluent la rapidité, la transparence, la sécurité et la centralisation des données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

L'objectif principal est de digitaliser entièrement le processus. Les objectifs incluent la rapidité, la transparence, la sécurité et la centralisation des données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

L'objectif principal est de digitaliser entièrement le processus. Les objectifs incluent la rapidité, la transparence, la sécurité et la centralisation des données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

L'objectif principal est de digitaliser entièrement le processus. Les objectifs incluent la rapidité, la transparence, la sécurité et la centralisation des données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

3. Architecture du système

Le système repose sur une architecture en 3 couches : frontend, backend et base de données. Cette séparation permet une meilleure maintenance et évolutivité. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système repose sur une architecture en 3 couches : frontend, backend et base de données. Cette séparation permet une meilleure maintenance et évolutivité. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système repose sur une architecture en 3 couches : frontend, backend et base de données. Cette séparation permet une meilleure maintenance et évolutivité. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système repose sur une architecture en 3 couches : frontend, backend et base de données. Cette séparation permet une meilleure maintenance et évolutivité. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système repose sur une architecture en 3 couches : frontend, backend et base de données. Cette séparation permet une meilleure maintenance et évolutivité. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système repose sur une architecture en 3 couches : frontend, backend et base de données. Cette séparation permet une meilleure maintenance et évolutivité. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système repose sur une architecture en 3 couches : frontend, backend et base de données. Cette séparation permet une meilleure maintenance et évolutivité. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système repose sur une architecture en 3 couches : frontend, backend et base de données. Cette séparation permet une meilleure maintenance et évolutivité. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

4. Détails Backend

Le backend est développé avec Node.js et Express. Il gère la logique métier, les règles de validation et la communication avec la base de données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le backend est développé avec Node.js et Express. Il gère la logique métier, les règles de validation et la communication avec la base de données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le backend est développé avec Node.js et Express. Il gère la logique métier, les règles de validation et la communication avec la base de données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le backend est développé avec Node.js et Express. Il gère la logique métier, les règles de validation et la communication avec la base de données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le backend est développé avec Node.js et Express. Il gère la logique métier, les règles de validation et la communication avec la base de données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le backend est développé avec Node.js et Express. Il gère la logique métier, les règles de validation et la communication avec la base de données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le backend est développé avec Node.js et Express. Il gère la logique métier, les règles de validation et la communication avec la base de données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le backend est développé avec Node.js et Express. Il gère la logique métier, les règles de validation et la communication avec la base de données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

5. Base de données

PostgreSQL est utilisé pour garantir la cohérence des données et gérer les relations complexes entre utilisateurs, projets et évaluations. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

PostgreSQL est utilisé pour garantir la cohérence des données et gérer les relations complexes entre utilisateurs, projets et évaluations. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

PostgreSQL est utilisé pour garantir la cohérence des données et gérer les relations complexes entre utilisateurs, projets et évaluations. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

PostgreSQL est utilisé pour garantir la cohérence des données et gérer les relations complexes entre utilisateurs, projets et évaluations. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

PostgreSQL est utilisé pour garantir la cohérence des données et gérer les relations complexes entre utilisateurs, projets et évaluations. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

PostgreSQL est utilisé pour garantir la cohérence des données et gérer les relations complexes entre utilisateurs, projets et évaluations. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

PostgreSQL est utilisé pour garantir la cohérence des données et gérer les relations complexes entre utilisateurs, projets et évaluations. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

PostgreSQL est utilisé pour garantir la cohérence des données et gérer les relations complexes entre utilisateurs, projets et évaluations. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

6. API REST

L'API permet la communication entre le frontend et le backend. Elle utilise le format JSON et suit les standards REST. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

L'API permet la communication entre le frontend et le backend. Elle utilise le format JSON et suit les standards REST. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

L'API permet la communication entre le frontend et le backend. Elle utilise le format JSON et suit les standards REST. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

L'API permet la communication entre le frontend et le backend. Elle utilise le format JSON et suit les standards REST. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

L'API permet la communication entre le frontend et le backend. Elle utilise le format JSON et suit les standards REST. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

L'API permet la communication entre le frontend et le backend. Elle utilise le format JSON et suit les standards REST. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

L'API permet la communication entre le frontend et le backend. Elle utilise le format JSON et suit les standards REST. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

L'API permet la communication entre le frontend et le backend. Elle utilise le format JSON et suit les standards REST. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

7. Sécurité

Le système utilise JWT pour l'authentification et bcrypt pour le hash des mots de passe. Les accès sont contrôlés selon les rôles. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système utilise JWT pour l'authentification et bcrypt pour le hash des mots de passe. Les accès sont contrôlés selon les rôles. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système utilise JWT pour l'authentification et bcrypt pour le hash des mots de passe. Les accès sont contrôlés selon les rôles. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système utilise JWT pour l'authentification et bcrypt pour le hash des mots de passe. Les accès sont contrôlés selon les rôles. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système utilise JWT pour l'authentification et bcrypt pour le hash des mots de passe. Les accès sont contrôlés selon les rôles. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système utilise JWT pour l'authentification et bcrypt pour le hash des mots de passe. Les accès sont contrôlés selon les rôles. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système utilise JWT pour l'authentification et bcrypt pour le hash des mots de passe. Les accès sont contrôlés selon les rôles. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système utilise JWT pour l'authentification et bcrypt pour le hash des mots de passe. Les accès sont contrôlés selon les rôles. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

8. Modélisation

Les principales entités sont : User, Projet, Evaluation, Subvention. Les relations sont définies pour assurer l'intégrité des données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Les principales entités sont : User, Projet, Evaluation, Subvention. Les relations sont définies pour assurer l'intégrité des données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Les principales entités sont : User, Projet, Evaluation, Subvention. Les relations sont définies pour assurer l'intégrité des données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Les principales entités sont : User, Projet, Evaluation, Subvention. Les relations sont définies pour assurer l'intégrité des données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Les principales entités sont : User, Projet, Evaluation, Subvention. Les relations sont définies pour assurer l'intégrité des données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Les principales entités sont : User, Projet, Evaluation, Subvention. Les relations sont définies pour assurer l'intégrité des données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Les principales entités sont : User, Projet, Evaluation, Subvention. Les relations sont définies pour assurer l'intégrité des données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Les principales entités sont : User, Projet, Evaluation, Subvention. Les relations sont définies pour assurer l'intégrité des données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

9. UI/UX

L'application mobile permet aux candidats de soumettre leurs projets. L'application web permet aux administrateurs de gérer les dossiers. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

L'application mobile permet aux candidats de soumettre leurs projets. L'application web permet aux administrateurs de gérer les dossiers. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

L'application mobile permet aux candidats de soumettre leurs projets. L'application web permet aux administrateurs de gérer les dossiers. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

L'application mobile permet aux candidats de soumettre leurs projets. L'application web permet aux administrateurs de gérer les dossiers. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

L'application mobile permet aux candidats de soumettre leurs projets. L'application web permet aux administrateurs de gérer les dossiers. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

L'application mobile permet aux candidats de soumettre leurs projets. L'application web permet aux administrateurs de gérer les dossiers. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

L'application mobile permet aux candidats de soumettre leurs projets. L'application web permet aux administrateurs de gérer les dossiers. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

L'application mobile permet aux candidats de soumettre leurs projets. L'application web permet aux administrateurs de gérer les dossiers. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

10. Déploiement

Le système est déployé sur un serveur cloud avec une base de données PostgreSQL. L'application mobile est distribuée en APK. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système est déployé sur un serveur cloud avec une base de données PostgreSQL. L'application mobile est distribuée en APK. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système est déployé sur un serveur cloud avec une base de données PostgreSQL. L'application mobile est distribuée en APK. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système est déployé sur un serveur cloud avec une base de données PostgreSQL. L'application mobile est distribuée en APK. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système est déployé sur un serveur cloud avec une base de données PostgreSQL. L'application mobile est distribuée en APK. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système est déployé sur un serveur cloud avec une base de données PostgreSQL. L'application mobile est distribuée en APK. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système est déployé sur un serveur cloud avec une base de données PostgreSQL. L'application mobile est distribuée en APK. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système est déployé sur un serveur cloud avec une base de données PostgreSQL. L'application mobile est distribuée en APK. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

11. Étude comparative backend

Node.js, Laravel et Django ont été comparés. Node.js a été choisi pour sa performance et sa gestion asynchrone. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Node.js, Laravel et Django ont été comparés. Node.js a été choisi pour sa performance et sa gestion asynchrone. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Node.js, Laravel et Django ont été comparés. Node.js a été choisi pour sa performance et sa gestion asynchrone. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Node.js, Laravel et Django ont été comparés. Node.js a été choisi pour sa performance et sa gestion asynchrone. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Node.js, Laravel et Django ont été comparés. Node.js a été choisi pour sa performance et sa gestion asynchrone. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Node.js, Laravel et Django ont été comparés. Node.js a été choisi pour sa performance et sa gestion asynchrone. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Node.js, Laravel et Django ont été comparés. Node.js a été choisi pour sa performance et sa gestion asynchrone. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Node.js, Laravel et Django ont été comparés. Node.js a été choisi pour sa performance et sa gestion asynchrone. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

12. Étude comparative mobile

Flutter et React Native ont été comparés. Flutter a été choisi pour ses performances et son rendu natif. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Flutter et React Native ont été comparés. Flutter a été choisi pour ses performances et son rendu natif. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Flutter et React Native ont été comparés. Flutter a été choisi pour ses performances et son rendu natif. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Flutter et React Native ont été comparés. Flutter a été choisi pour ses performances et son rendu natif. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Flutter et React Native ont été comparés. Flutter a été choisi pour ses performances et son rendu natif. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Flutter et React Native ont été comparés. Flutter a été choisi pour ses performances et son rendu natif. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Flutter et React Native ont été comparés. Flutter a été choisi pour ses performances et son rendu natif. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Flutter et React Native ont été comparés. Flutter a été choisi pour ses performances et son rendu natif. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

13. Étude comparative base de données

PostgreSQL et MongoDB ont été comparés. PostgreSQL a été retenu pour sa structure relationnelle. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

PostgreSQL et MongoDB ont été comparés. PostgreSQL a été retenu pour sa structure relationnelle. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

PostgreSQL et MongoDB ont été comparés. PostgreSQL a été retenu pour sa structure relationnelle. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

PostgreSQL et MongoDB ont été comparés. PostgreSQL a été retenu pour sa structure relationnelle. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

PostgreSQL et MongoDB ont été comparés. PostgreSQL a été retenu pour sa structure relationnelle. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

PostgreSQL et MongoDB ont été comparés. PostgreSQL a été retenu pour sa structure relationnelle. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

PostgreSQL et MongoDB ont été comparés. PostgreSQL a été retenu pour sa structure relationnelle. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

PostgreSQL et MongoDB ont été comparés. PostgreSQL a été retenu pour sa structure relationnelle. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

14. Scalabilité

Le système peut évoluer grâce à une architecture modulaire et un déploiement cloud. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système peut évoluer grâce à une architecture modulaire et un déploiement cloud. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système peut évoluer grâce à une architecture modulaire et un déploiement cloud. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système peut évoluer grâce à une architecture modulaire et un déploiement cloud. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système peut évoluer grâce à une architecture modulaire et un déploiement cloud. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système peut évoluer grâce à une architecture modulaire et un déploiement cloud. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système peut évoluer grâce à une architecture modulaire et un déploiement cloud. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Le système peut évoluer grâce à une architecture modulaire et un déploiement cloud. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

15. Défis techniques

Gestion des fichiers, sécurité, performances et synchronisation des données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Gestion des fichiers, sécurité, performances et synchronisation des données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Gestion des fichiers, sécurité, performances et synchronisation des données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Gestion des fichiers, sécurité, performances et synchronisation des données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Gestion des fichiers, sécurité, performances et synchronisation des données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Gestion des fichiers, sécurité, performances et synchronisation des données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Gestion des fichiers, sécurité, performances et synchronisation des données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

Gestion des fichiers, sécurité, performances et synchronisation des données. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

16. Conclusion

La solution proposée permet une gestion moderne, efficace et sécurisée des subventions. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

La solution proposée permet une gestion moderne, efficace et sécurisée des subventions. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

La solution proposée permet une gestion moderne, efficace et sécurisée des subventions. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

La solution proposée permet une gestion moderne, efficace et sécurisée des subventions. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

La solution proposée permet une gestion moderne, efficace et sécurisée des subventions. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

La solution proposée permet une gestion moderne, efficace et sécurisée des subventions. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

La solution proposée permet une gestion moderne, efficace et sécurisée des subventions. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.

La solution proposée permet une gestion moderne, efficace et sécurisée des subventions. Cette approche garantit la robustesse du système et permet une évolution future sans refonte majeure.